

Voter ID / EPIC Assistant

A Retrieval-Augmented Generation (RAG) Chatbot for Government Services

Team: Vibe Code

Param Modi (2025201087)

Meet Ghelani (2025201096)

Pratee Tiwari (2025201037)

May 6, 2026

GitHub: https://github.com/meet6868/SMAI_ASS_3

Live App: <https://ndt6ujti6anrngsjgcdllp.streamlit.app/>

Abstract

This report presents a Retrieval-Augmented Generation (RAG) based chatbot designed to assist users in accessing Indian voter ID (EPIC) related services. The system retrieves relevant information from official government PDFs and generates grounded responses using large language models. It avoids model training by leveraging semantic search and external APIs. The final system is deployed as an interactive Streamlit application and supports both English and Hinglish queries.

1 Introduction

Accessing government services often requires users to go through multiple documents that are not easy to navigate. In the case of voter ID services, users must manually read different forms and instructions, which can be time-consuming and confusing.

To simplify this process, we developed a chatbot that allows users to ask questions in natural language. Instead of searching manually, the system retrieves relevant information from official documents and presents it in a clear and concise manner. Each answer is supported with citations, ensuring that the information is trustworthy.

2 Data

The system uses official voter-related forms as its data source. These include Form 6, Form 6A, Form 7, and Form 8, which cover registration, correction, and deletion processes.

The PDFs are processed through a pipeline where text is extracted, cleaned, and divided into smaller chunks. Each chunk retains metadata such as file name and page number. This metadata is later used to provide citations in the final response.

3 Method

3.1 System Overview

The system follows a Retrieval-Augmented Generation pipeline, where retrieval and generation are clearly separated. The overall workflow is shown in Figure 1.

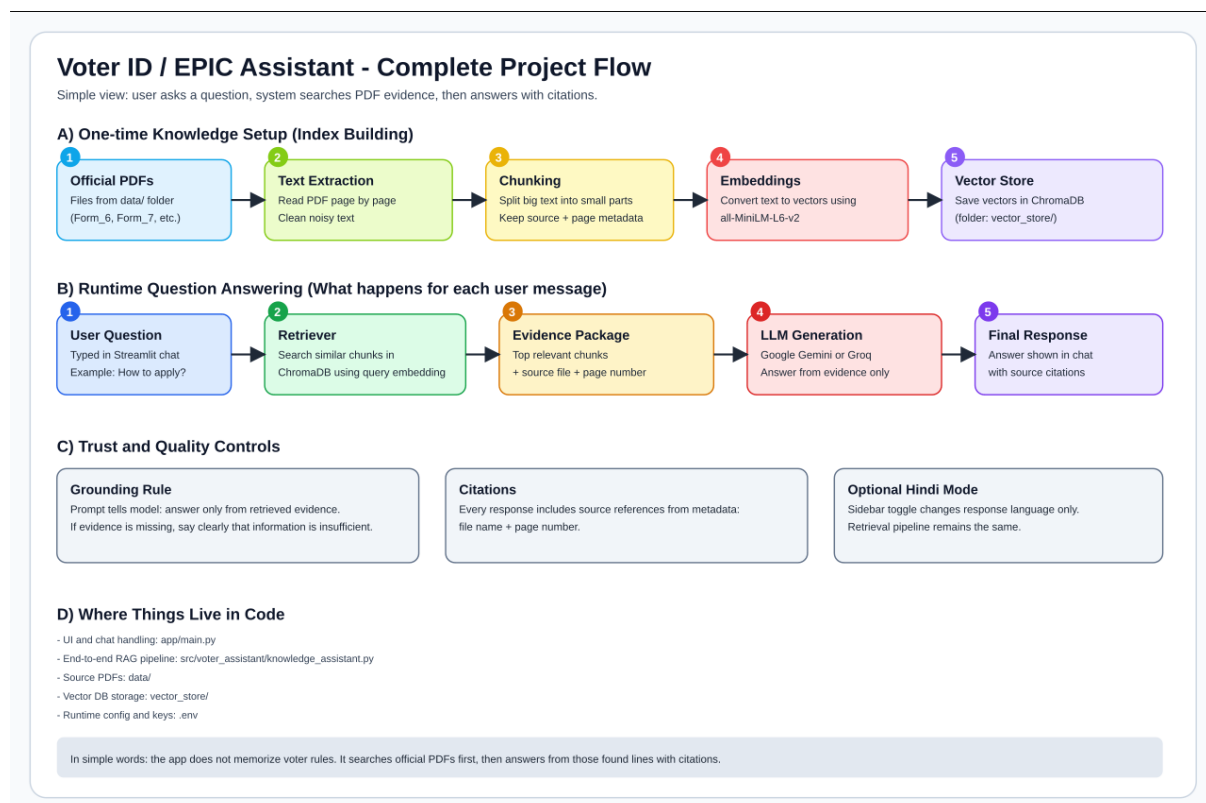


Figure 1: RAG pipeline showing document processing, embedding, retrieval, and answer generation stages.

The pipeline begins by converting PDFs into text and dividing them into overlapping chunks. These chunks are then converted into embeddings and stored in a vector database. During query time, the system retrieves the most relevant chunks and passes them to a language model, which generates the final answer.

3.2 Embedding and Retrieval

We use the all-MiniLM-L6-v2 model to generate embeddings. This model is efficient and suitable for CPU-based environments. The embeddings are stored in ChromaDB, which enables fast similarity search.

At runtime:

- The user query is converted into an embedding
- Similar chunks are retrieved using cosine similarity
- Top- k relevant chunks are selected as context

3.3 Answer Generation

The retrieved chunks are passed to a language model (Gemini or Groq Llama-3). The prompt ensures that the model generates answers strictly based on the provided context. The output also includes citations pointing to the source documents.

3.4 Query Handling

To improve usability, the system includes a simple query expansion mechanism. For example, informal queries such as “name change karna hai” are expanded with voter-related context to improve retrieval accuracy.

3.5 Key Insights

From our experiments and usage, we observed several important behaviors of the system:

- **Retrieval dominates performance:** The quality of the final answer depends heavily on the retrieved chunks. If retrieval fails, the language model produces weak or incomplete answers.
- **Chunk size trade-off:** Smaller chunks improve retrieval precision but sometimes lose context, while larger chunks preserve context but may reduce retrieval accuracy.
- **Query expansion is effective:** Simple keyword-based expansion significantly improves performance for Hinglish queries without requiring multilingual embeddings.
- **LLM variability:** Different LLM providers (Gemini vs Groq) show noticeable differences in response style and latency, even when given the same context.
- **Latency bottleneck:** Most of the response time comes from the generation step rather than retrieval.

4 User Interface

The system is deployed using Streamlit and provides a chat-based interface. Users can enter queries and receive responses in real time.

Key features include:

- Chat interface using `st.chat_input`
- Display of answers with citations

- Optional Hindi response mode
- Text-to-speech support for responses

4.1 Interface Snapshots

Figure 2 shows the initial loading state of the deployed Streamlit app.

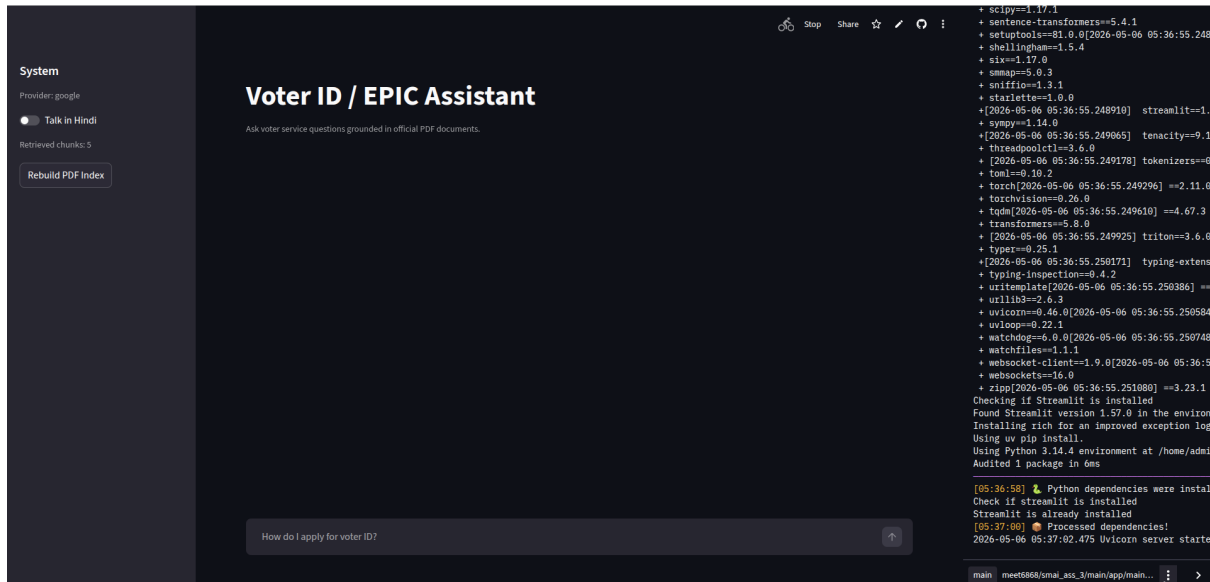


Figure 2: Streamlit app loading state on deployment.

Figure 3 illustrates an English query with a grounded response and citations.

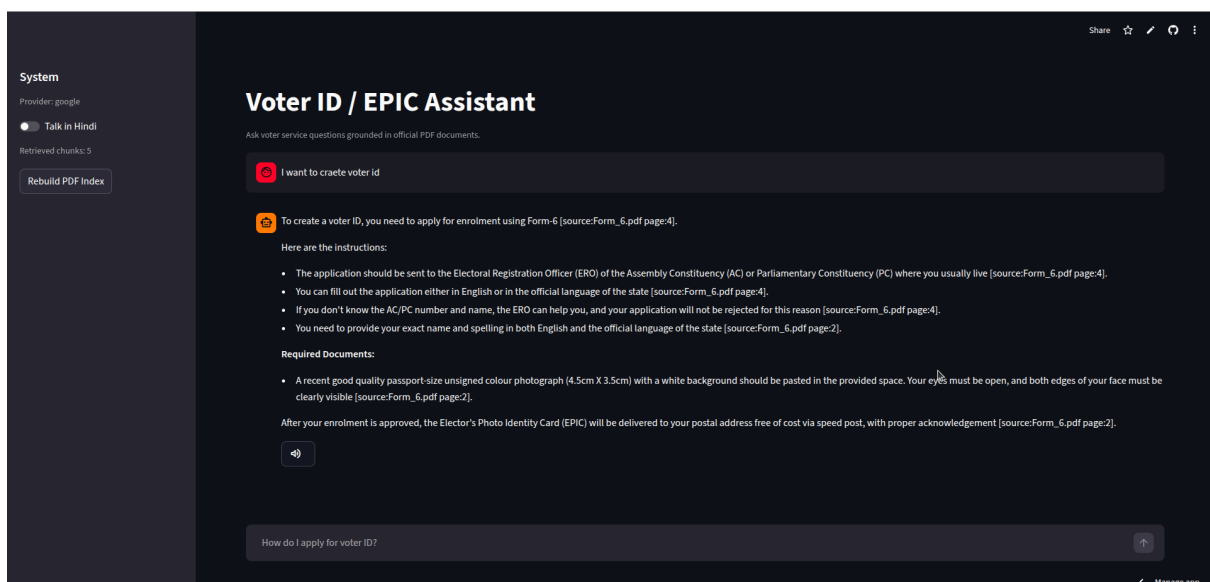


Figure 3: English query and answer with sources.

Figure 4 shows a Hinglish query where the assistant returns an English response.

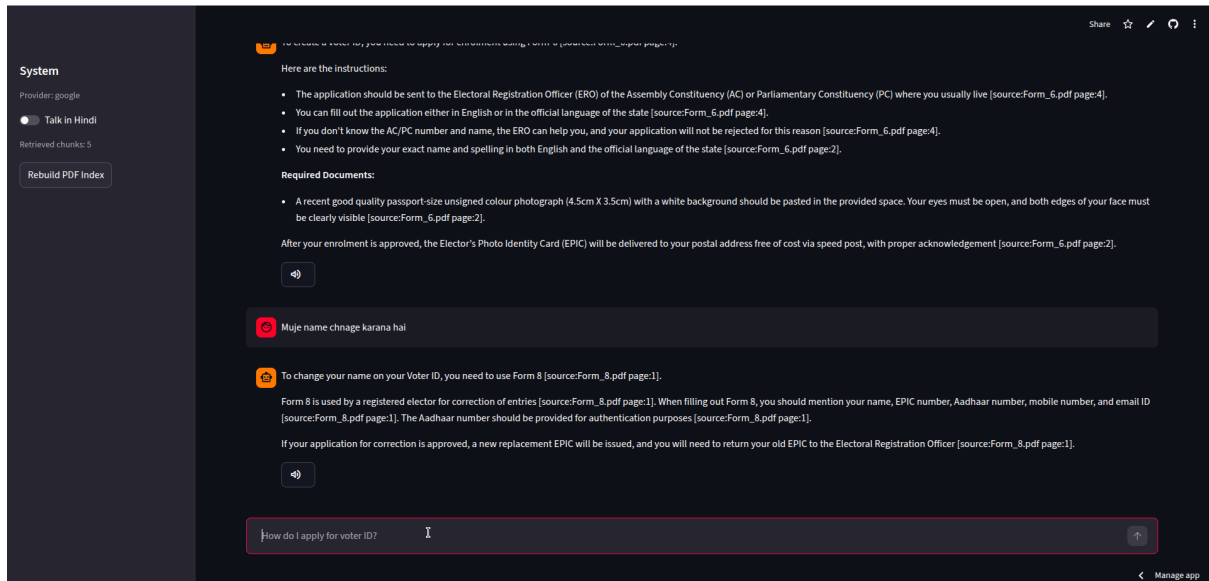


Figure 4: Hinglish query with English response.

Figure 5 demonstrates a Hinglish query with Hindi response mode enabled.

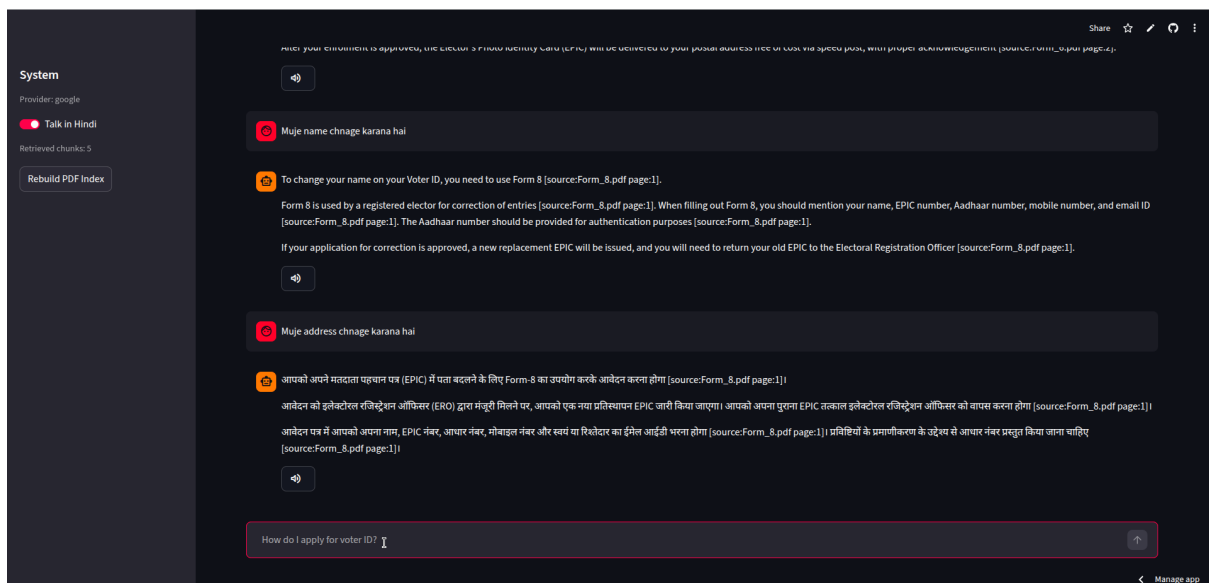


Figure 5: Hinglish query with Hindi response.

5 Results

The system was evaluated using several sample queries related to voter ID services. It was observed that the chatbot retrieves relevant information and generates useful responses.

In most cases:

- Retrieved documents were relevant to the query

- Responses were grounded in the retrieved context
- Citations correctly matched the source documents

The system also handled simple Hinglish queries reasonably well due to query expansion.

5.1 Performance Measurements

We instrumented the application to record retrieval time, generation time, end-to-end latency, and response sizes. The Streamlit app logs each query to a CSV file with prompt/answer size estimates and process memory usage. Table 1 reports the measured values from `data/metrics.csv`.

Query	Retrieve (ms)	Generate (ms)	End-to-end (ms)	Prompt chars	Answer chars	RSS (MB)
I want to create Voter Id	26.59	6841.86	8879.71	4894	765	1276.04
Muje name and address change karana hai	25.10	6987.82	7012.96	4917	798	1277.55
Muje dob change karani hai	25.16	15174.23	15199.42	4966	414	1278.48
Muje mobile no update karana hai	25.02	27217.31	27242.38	4968	421	1278.80
What is EPIC in voter ID?	28.22	4980.10	5008.38	4892	381	1278.94
How to delete my name from voter list?	23.13	11578.59	11601.76	4914	1019	1279.20

Table 1: Latency and resource measurements collected from the deployed app.

6 Ablation Study

Due to time and resource constraints, we did not perform a full quantitative ablation study across multiple configurations. Instead, we fixed a single pipeline setup that provided stable and reliable performance for our use case.

We selected the `all-MiniLM-L6-v2` embedding model due to its efficiency and good semantic performance in general-purpose tasks. The chunk size and overlap were chosen empirically to balance context preservation and retrieval accuracy.

We also fixed the number of retrieved chunks ($k = 4$) to avoid excessive context while still providing sufficient information to the language model.

Although no systematic ablation experiments were conducted, informal testing indicated that:

- Very small chunk sizes reduced context quality
- Very large chunks reduced retrieval precision
- Increasing k beyond 4 did not significantly improve answers but increased latency

Future work can include controlled experiments to evaluate the impact of chunk size, retrieval depth, and different embedding models.

7 Limitations

Despite good performance, the system has some limitations:

- Limited dataset (only a few PDF forms)
- Dependence on external LLM APIs
- Limited multilingual support
- Requires re-indexing when documents change

8 Conclusion

This project demonstrates how a RAG-based system can be used to build an effective domain-specific chatbot without training a model. By combining retrieval with language generation, the system provides accurate and explainable responses. The deployed application highlights the practical usefulness of such systems in improving access to government services.

Team Contribution

- **Param Modi (2025201087)**: Developed the core RAG pipeline
- **Meet Ghelani (2025201096)**: Deployment and report writing
- **Pratee Tiwari (2025201037)**: Streamlit UI development

Team Discussion

All team members contributed to system design decisions, including architecture, model selection, and integration. The final system is a result of collaborative development and iterative improvements.

9 References

References

- [1] Lewis et al., Retrieval-Augmented Generation, NeurIPS 2020
- [2] Reimers & Gurevych, Sentence-BERT, EMNLP 2019
- [3] ChromaDB Documentation
- [4] <https://www.nvsp.in>
- [5] <https://eci.gov.in>